PrediCare TPA — Règles Frontend (Next.js)

Référentiel de conception et d'ingénierie frontend · Version 1.0 · Octobre 2026

0. Comment utiliser ce document

Ce document est **opposable** : toute PR frontend doit le respecter. Il est découpé pour être placé dans le dépôt (`docs/FRONTEND_RULES.md`) et relu à chaque revue de code.

Vocabulaire : **DOIT** = obligatoire (bloquant en revue) · **DEVRAIT** = fortement recommandé (dérogation justifiée dans la PR) · **PEUT** = optionnel.

Le contexte produit vient du cahier des charges : plateforme de détection de fraude médicale pour un TPA, 4 rôles (professionnel de santé, assuré, gestionnaire, administrateur), alertes avec explicabilité SHAP, assistant RAG agentique, dashboard BI, écran d'administration (seuil, versions de modèle, supervision).

1. Principes directeurs et personnalité

Personnalité : « le clinicien vigilant ». Sobre, précis, rassurant, jamais alarmiste. L'interface inspire la confiance d'un dossier médical bien tenu et l'acuité d'un analyste financier.

Principe	Traduction concrète
Clarté avant décoration	Un écran = une tâche principale. Pas d'animation gratuite, pas de dégradés décoratifs.
Le risque se lit en 2 secondes	Score, niveau, raison principale visibles sans scroller dans toute liste d'alertes.
L'IA explique, l'humain décide	Chaque alerte affiche le « pourquoi » (SHAP) et exige une décision humaine motivée (AI Act, supervision humaine).
Calme sous pression	Le gestionnaire traite des dizaines d'alertes : densité maîtrisée, raccourcis clavier, états stables, pas de saut de mise en page.
Données de santé = prudence	Masquage par défaut des données sensibles, aucune fuite dans l'URL, les logs ou le cache navigateur.

Principe	Traduction concrète
Honnêteté	Toujours dire quand c'est l'IA, quand une donnée est absente, quand une erreur est survenue.

Signature visuelle : un motif discret de *ligne de pouls* (logo, états vides, écran de connexion) et un **anneau de score** (jauge circulaire) réutilisé partout où un risque est affiché. Aucun autre ornement.

Ton rédactionnel (français, vouvoiement) : phrases courtes, verbes d'action, pas de jargon ML dans les portails assuré et professionnel.

À éviter	À écrire
« Erreur 422 »	« Cette facture ne peut pas être traitée : le code d'acte 'X12' n'existe pas dans le référentiel. »
« Fraude détectée ! »	« Alerte de risque élevé (score 0,91). Vérification recommandée. »
« Succès »	« Décision enregistrée. Le prestataire sera notifié. »

Vocabulaire : on parle d'**alerte** et de **risque**, jamais d'« accusation » ni de « fraudeur ». Un score est un indice de priorisation, pas un verdict.

2. Normes de référence

Domaine	Norme	Application
Accessibilité	**WCAG 2.2 niveau AA** et **RGAA 4.1** (référentiel français)	Contrastes, clavier, lecteurs d'écran, déclaration d'accessibilité
Ergonomie	**ISO 9241-210**, heuristiques de **Nielsen**	Conception centrée utilisateur, tests avec chaque rôle
Sécurité web	**OWASP ASVS niveau 2**, OWASP Top 10, OWASP LLM Top 10	Auth, CSP, injection de prompt, rendu des sorties LLM
Données personnelles	**RGPD** (minimisation, privacy by design)	Pseudonymisation visible dans l'UI, consentement, journal d'audit

Domaine	Norme	Application
IA	**EU AI Act** (système à haut risque : art. 13 transparence, art. 14 supervision humaine)	Explicabilité, mention IA, décision humaine, contestation
Performance	**Core Web Vitals**	LCP < 2,5 s · INP < 200 ms · CLS < 0,1
Code	TypeScript strict, ESLint, Prettier, **Conventional Commits**, **SemVer**	CI bloquante
Documentation	**ADR** (Architecture Decision Records), Keep a Changelog	`docs/adr/`, `CHANGELOG.md`

3. Stack technique

Besoin	Choix	Règle
Framework	**Next.js (App Router)** + React	Server Components par défaut ; `"use client"` uniquement pour l'interactivité
Langage	**TypeScript** `strict: true`	`any` interdit (règle ESLint)
Style	**Tailwind CSS** + **shadcn/ui** (Radix UI)	Aucune valeur de couleur codée en dur : uniquement des tokens
Données serveur	**TanStack Query** (client) + `fetch` côté serveur	Pas d'état serveur dans un store global
Formulaires	**react-hook-form** + **zod**	Même schéma zod pour validation et types
Graphiques	**Recharts** (dashboard) + composant SHAP maison	Toujours un tableau de données équivalent accessible
Client API	Généré par **openapi-typescript** depuis l'OpenAPI de FastAPI	Jamais d'appel `fetch` « à la main » hors de `lib/api`
i18n	**next-intl**, français par défaut	Aucun texte en dur dans les composants

Besoin	Choix	Règle
Auth	Cookies **httpOnly + Secure + SameSite=Lax** émis par l'API	Pas de token dans `localStorage`
Tests	**Vitest**, Testing Library, **Playwright**, **axe-core**	Voir section 15
Catalogue	**Storybook**	Tout composant partagé a une story
Packaging	`output: "standalone"`, image Docker multi-stage, utilisateur non-root	Démarre via `docker compose up`

4. Architecture du projet

Organisation **par fonctionnalité** (feature-based) plutôt que par type de fichier.

```
frontend/
├─ src/
│  ├─ app/
│  │  ├─ (auth)/login, mot-de-passe-oublie
│  │  ├─ (pro)/            # professionnel de santé
│  │  ├─ (assure)/         # assuré
│  │  ├─ (gestionnaire)/   # alertes, factures, ordonnances, dashboard
│  │  ├─ (admin)/          # seuil, modèles, supervision, utilisateurs, audit
│  │  ├─ api/health/route.ts
│  │  ├─ layout.tsx, error.tsx, not-found.tsx, loading.tsx
│  ├─ features/
│  │  ├─ invoices/  alerts/  prescriptions/  assistant/
│  │  ├─ dashboard/  models/  monitoring/  audit/  users/
│  │  └─ (chaque feature : components/ hooks/ schemas/ api.ts index.ts)
│  ├─ components/
│  │  ├─ ui/        # primitives shadcn (Button, Input, Dialog…)
│  │  └─ domain/    # RiskBadge, ScoreRing, ShapChart, KpiCard…
│  ├─ lib/ (api/, auth/, env.ts, format.ts, rbac.ts, logger.ts)
│  ├─ styles/tokens.css
│  ├─ i18n/fr.json
│  └─ middleware.ts   # auth + RBAC
├─ tests/ (unit/, e2e/, a11y/)
```

```
├─ .storybook/
├─ docs/ (adr/, user-guide/, FRONTEND_RULES.md)
├─ .env.example
└─ Dockerfile
```

Règles d'architecture

- **DOIT** : une feature n'importe jamais le contenu interne d'une autre feature, uniquement son `index.ts`.
- **DOIT** : `components/ui` ne contient aucune logique métier ; `components/domain` ne fait aucun appel réseau.
- **DOIT** : pages `page.tsx` fines (assemblage) ; la logique vit dans `features/`.
- **DOIT** : chaque route a ses états `loading.tsx` et `error.tsx`.
- **DEVRAIT** : fichier ⩽ 250 lignes, composant ⩽ 150 lignes.
- Nommage : composants `PascalCase`, hooks `useXxx`, fichiers `kebab-case`, constantes `UPPER_SNAKE`.

5. Design tokens

Les tokens sont définis une seule fois dans `styles/tokens.css` et exposés à Tailwind. **Aucun hexadécimal en dehors de ce fichier.**

5.1 Palette

Rôle	Nom	Clair	Usage
Marque / action	`primary`	`#0B7A75` (teal « soin »)	Boutons principaux, liens, focus de marque
Marque survol	`primary-hover`	`#086560`	Hover/active
Marque douce	`primary-soft`	`#E3F4F2`	Fonds de sélection, badges de marque
Structure	`navy`	`#0B2545`	Barre latérale, en-têtes de zones sombres
Fond application	`background`	`#F6F8FB`	Arrière-plan des pages
Surface	`surface`	`#FFFFFF`	Cartes, tableaux, dialogues

Rôle	Nom	Clair	Usage
Surface atténuée	`surface-muted`	`#EEF2F7`	En-têtes de tableau, zones secondaires
Bordure	`border`	`#DDE3EC`	Séparateurs, champs
Texte	`foreground`	`#0E1A2B`	Texte principal
Texte secondaire	`muted-foreground`	`#526074`	Légendes, aides
Information	`info`	`#1D5FD1`	Messages neutres, anneau de focus

Sémantique du risque (réservée au risque, jamais décorative)

Niveau	Texte	Fond doux	Règle
Faible	`#167A4F`	`#E4F5EC`	score < 0,5
Moyen	`#9A5B00`	`#FFF3DB`	0,5 ⩽ score < seuil
Élevé	`#C2410C`	`#FFE9DD`	score ⩾ seuil (0,8 par défaut, configurable en admin)
Erreur système	`#B42318`	`#FDE7E5`	Échecs techniques, validations (≠ risque)
Invalide / hors périmètre	`#526074`	`#EEF2F7`	Facture rejetée, donnée absente

```
:root {
  --background:#F6F8FB; --surface:#FFFFFF; --surface-muted:#EEF2F7;
  --border:#DDE3EC; --foreground:#0E1A2B; --muted-foreground:#526074;
  --primary:#0B7A75; --primary-hover:#086560; --primary-soft:#E3F4F2;
  --primary-foreground:#FFFFFF; --navy:#0B2545; --info:#1D5FD1;
  --risk-low:#167A4F;    --risk-low-bg:#E4F5EC;
  --risk-medium:#9A5B00; --risk-medium-bg:#FFF3DB;
  --risk-high:#C2410C;   --risk-high-bg:#FFE9DD;
  --danger:#B42318;      --danger-bg:#FDE7E5;
  --radius-sm:6px; --radius-md:8px; --radius-lg:12px;
}
[data-theme="dark"] {
```

```
  --background:#0A1220; --surface:#111B2E; --surface-muted:#182540;
  --border:#25324D; --foreground:#E8EEF7; --muted-foreground:#9AA9C0;
  --primary:#35B8AF; --primary-hover:#5CCBC3; --primary-soft:#12323A;
  --primary-foreground:#04201F;
  --risk-low:#4ADE9A;    --risk-low-bg:#10301F;
  --risk-medium:#F5B84A; --risk-medium-bg:#3A2B0A;
  --risk-high:#FF8A5B;   --risk-high-bg:#3D1C10;
}
```

Règles couleur

- **DOIT** : contraste texte ⩾ 4,5:1 (⩾ 3:1 pour texte large et composants UI). Un test automatisé en CI vérifie toutes les paires de tokens.
- **DOIT** : **ne jamais transmettre une information par la couleur seule** : tout niveau de risque combine couleur + icône + libellé (« Élevé »).
- **DOIT** : le rouge/orange est **réservé au risque et aux erreurs**. Le teal est la seule couleur d'action.
- **DOIT** : dans le graphique SHAP, contribution qui augmente le risque = orange `risk-high` avec « + », qui le diminue = bleu `info` avec « − » (pas rouge/vert : daltonisme).
- **DEVRAIT** : thème sombre proposé aux gestionnaires (sessions longues) ; clair par défaut pour assuré et professionnel.

5.2 Typographie

- **IBM Plex Sans** (interface, titres semi-bold) et **IBM Plex Mono** (identifiants, références de facture, codes d'actes), chargées via `next/font` (auto-hébergées, pas de CDN : RGPD).
- Chiffres **tabulaires** (`font-variant-numeric: tabular-nums`) pour tous montants, scores et colonnes numériques, alignés à droite.
- Échelle : 12 · 14 · 16 · 20 · 24 · 30 · 36. Corps **14 px** en back-office (gestionnaire, admin), **16 px** sur les portails assuré et professionnel.
- Interligne 1,5 (texte), 1,25 (titres). Longueur de ligne ⩽ 75 caractères.

5.3 Espacement, forme, mouvement

- Grille de **4 px** (4, 8, 12, 16, 24, 32, 48). Rayons 6/8/12 px. Ombres légères, bordures préférées aux ombres.
- Cible tactile ⩾ **44×44 px** sur portails assuré/professionnel (⩾ 24 px minimum WCAG 2.2 en back-office).

- Mouvement 150–200 ms, `ease-out`, uniquement pour expliquer un changement d'état. Respect de `prefers-reduced-motion`.
- Icônes : **lucide-react**, trait 1,5 px, toujours accompagnées d'un libellé ou d'un `aria-label`.

6. Catalogue de composants

6.1 Primitives (`components/ui`)

Button (primary, secondary, ghost, destructive ; états loading/disabled) · Input, Select, Combobox, DatePicker, Textarea · Checkbox, Radio, Switch · Dialog, AlertDialog, Sheet, Popover, Tooltip · Tabs, Breadcrumb, Pagination · Table (tri, sticky header) · Badge · Toast · Skeleton · Alert · Card · Progress.

Règles : un seul bouton primaire par zone ; action destructive = `AlertDialog` avec libellé explicite (« Restaurer la version 1.3.0 ») ; le focus est toujours visible (anneau 2 px `info`, offset 2 px).

6.2 Composants métier (`components/domain`)

Composant	Rôle	Règles clés
`ScoreRing`	Jauge circulaire du score 0–1	Valeur numérique au centre, texte alternatif, couleur par niveau, marque visuelle du seuil
`RiskBadge`	Niveau Faible/Moyen/Élevé	Icône + libellé + couleur ; jamais couleur seule
`ShapChart`	Contributions SHAP par variable (barres horizontales triées)	Top 8 par défaut + « Voir tout », tableau accessible équivalent, info-bulle en langage métier
`ExplanationSummary`	Phrase générée : « Montant 3,2× supérieur à la médiane pour cet acte »	Toujours au-dessus du graphique

Composant	Rôle	Règles clés
`AlertCard` / `AlertRow`	Alerte dans une file	Score, prestataire (pseudonyme), montant, date, motif principal, statut
`DecisionPanel`	Valider l'alerte / Écarter / Escalader	Motif **obligatoire** (≥ 10 caractères), confirmation, horodatage, trace d'audit
`InvoiceSummary`, `InvoiceLinesTable`	Détail facture et lignes d'actes	Lignes en anomalie surlignées + icône
`PrescriptionAnomalyPanel`	Doublon, incohérence prescription/pathologie	Lien vers les ordonnances comparées côte à côte
`InvoiceUploadDropzone`	Dépôt d'une facture ou d'un lot	Validation locale, rapport **par facture** (acceptée / rejetée + raison), jamais d'échec global opaque
`ClaimTimeline`	Suivi de remboursement (portail assuré)	Étapes datées, statut en langage clair, prochaine étape attendue
`ChatWindow`, `Message`, `SourceCitation`	Assistant RAG	Badge « Assistant IA », sources cliquables (contrat, article, page), indicateur de saisie, boutons 👍/👎, bouton « contacter un conseiller »
`KpiCard`	Indicateur de pilotage	Valeur, variation vs période précédente, unité, info-bulle de définition

Composant	Rôle	Règles clés
`ThresholdControl`	Réglage du seuil d'alerte (admin)	Slider + champ numérique, aperçu d'impact (« +12 alertes/jour estimées »), confirmation
`ModelVersionCard`, `RollbackDialog`	Versions du modèle	Version, date, métriques (recall, précision), statut actif, retour arrière en 2 étapes
`HealthStatus`, `MetricChart`, `DriftChart`	Supervision	Statut par service, latence p50/p95, volumes, erreurs, dérive (PSI) avec seuil visuel
`AuditLogTable`	Journal d'audit	Filtres acteur/action/date, export, lecture seule
`PiiField`	Donnée personnelle	Masquée par défaut (`J•••• D••••`), révélation explicite, journalisée
`EmptyState`, `ErrorState`, `PermissionDenied`	États	Cause + action suivante ; motif « pouls » en illustration

Règle générale : tout composant livre **ses 5 états** : chargement (skeleton aux dimensions finales), vide, erreur (avec « Réessayer »), partiel, nominal. Chaque composant partagé a une story Storybook, des props typées et un commentaire TSDoc.

7. Navigation et parcours par rôle

Coque applicative : barre latérale `navy` (gestionnaire/admin) ou barre supérieure (portails assuré/pro, mobile-first), fil d'Ariane, recherche globale (⌘K) en back-office, sélecteur de thème, menu utilisateur avec rôle affiché.

Rôle	Écrans	Objectif principal
Professionnel de santé	Déposer une facture · Mes factures et statuts · Corriger une facture rejetée · Aide	Soumettre et suivre sans appeler le centre
Assuré	Mes remboursements · Détail d'un dossier · Mes garanties · Assistant	Savoir où en est mon dossier et ce que couvre mon contrat
Gestionnaire	**File d'alertes** (priorisée par score) · Détail alerte (explication + facture + ordonnances + historique) · Ordonnances suspectes · Dashboard BI · Mes décisions	Traiter vite, décider juste, justifier
Administrateur	Configuration (seuil, canaux Slack/Email) · Modèles (versions, bascule) · Supervision (/health, métriques, dérive) · Utilisateurs et rôles · Journal d'audit · Sauvegardes (statut)	Exploiter et garder la maîtrise

Détail d'alerte (écran clé), disposition fixe : en-tête (ScoreRing, RiskBadge, statut) → résumé en une phrase → graphique SHAP → lignes de facture → ordonnances liées → historique du prestataire → `DecisionPanel` collant en bas. Raccourcis : `J/K` naviguer, `V` valider, `E` écarter, `?` aide.

RBAC : les routes sont filtrées par `middleware.ts` **et** vérifiées côté serveur et côté API. Masquer un lien ne protège rien. Un rôle non autorisé voit `PermissionDenied`, jamais une page vide.

8. Règles UX

1. **Hiérarchie** : un titre de page, une action primaire, le reste en secondaire.
2. **Tableaux** : pagination, tri et filtres **côté serveur**, état conservé dans l'URL (partageable, sans donnée personnelle), en-tête collant, colonnes numériques alignées à droite, densité réglable (confortable/compacte), export CSV.

3. **Formulaires** : labels visibles (jamais le placeholder seul), validation à la sortie du champ puis en direct, messages d'erreur précis avec solution, focus sur le premier champ en erreur, aucune perte de saisie après erreur.
4. **Feedback** : toute action donne un retour < 100 ms ; au-delà de 1 s, indicateur ; au-delà de 10 s, progression et possibilité de quitter. Mises à jour optimistes interdites pour les décisions sur alerte.
5. **Prévention d'erreur** : confirmation pour les actions irréversibles ou à fort impact (décision, retour arrière de modèle, changement de seuil), avec rappel de la conséquence.
6. **Reconnaissance plutôt que mémorisation** : filtres récents, valeurs par défaut sensées, libellés métier.
7. **Formats** : `Intl` avec locale `fr` ; montants avec devise **issue de la configuration** (pas codée en dur) ; dates `19 oct. 2026` + heure 24 h ; scores à 2 décimales (`0,91`) ; identifiants en police mono avec bouton copier.
8. **Responsive** : mobile-first pour assuré et professionnel ; back-office optimisé ⩾ 1280 px, utilisable ⩾ 768 px.
9. **Temps réel** : nouvelles alertes signalées par un bandeau « 3 nouvelles alertes — Actualiser » (SSE/polling), sans réordonner la liste sous le curseur.
10. **Assistant RAG** : message d'accueil précisant qu'il s'agit d'une IA et ce qu'elle peut faire ; suggestions de questions ; réponses sourcées ; refus clair hors périmètre ; passage possible à un humain ; jamais d'affichage d'informations d'un autre assuré.

9. Accessibilité (WCAG 2.2 AA / RGAA 4.1)

- **DOIT** : navigation 100 % clavier, ordre de tabulation logique, lien d'évitement « Aller au contenu », focus visible, pas de piège clavier.
- **DOIT** : HTML sémantique (`nav`, `main`, `table`, `th scope`, titres hiérarchisés) ; ARIA seulement si le natif est insuffisant.
- **DOIT** : alternatives textuelles pour graphiques (ScoreRing, SHAP, dashboard) : tableau de données ou description.
- **DOIT** : annonces `aria-live="polite"` pour toasts, résultats de dépôt et nouvelles alertes ; messages de l'assistant annoncés.
- **DOIT** : zoom 200 % sans perte, reflow à 320 px, pas de contenu uniquement au survol.
- **DOIT** : `axe-core` dans Playwright sur chaque route principale : 0 violation critique/sérieuse.

- **DOIT** : page « Déclaration d'accessibilité » accessible depuis le pied de page.

10. Données, API et état

- Client TypeScript **généré** depuis l'OpenAPI FastAPI ; régénération en CI, échec si le contrat diverge.
- Lecture serveur dans les Server Components quand possible ; TanStack Query pour l'interactif (`staleTime` explicite, clés normalisées par feature).
- Format d'erreur API unique `{code, message, details[]}` mappé à des messages utilisateur via `i18n`. Les erreurs de validation de facture sont affichées **ligne par ligne**.
- Timeouts, annulation (`AbortController`) et nouvelle tentative idempotente pour les lectures uniquement.
- Validation `zod` des réponses sensibles (scores, SHAP) : une réponse invalide produit un `ErrorState`, jamais un affichage faux.
- Variables d'environnement validées au démarrage par `lib/env.ts` (zod) ; seules les variables `NEXT_PUBLIC_*` non secrètes sont exposées. **Aucun secret dans le bundle client.**
- Seuil d'alerte, canaux et paramètres : **toujours lus depuis l'API de configuration**, jamais codés en dur (changement en direct visible sans redéploiement).

11. Sécurité, RGPD et AI Act côté interface

Sujet	Règle
Authentification	Cookies httpOnly/Secure/SameSite ; expiration d'inactivité (15 min back-office) avec avertissement ; déconnexion visible ; protection CSRF
Autorisation	RBAC dans middleware + serveur + API ; principe du moindre privilège
En-têtes	CSP stricte (nonce), `X-Content-Type-Options`, `Referrer-Policy: no-referrer`, `frame-ancestors 'none'`, HSTS

Sujet	Règle
Rendu LLM	Sortie de l'assistant rendue en **texte/Markdown assaini** ; `dangerouslySetInnerHTML` interdit ; liens sortants filtrés ; aucune exécution de contenu issu d'un document
Prompt injection	Limite de longueur et nettoyage de l'entrée côté UI (défense en profondeur, la protection réelle est côté serveur) ; message neutre en cas de requête bloquée
PII	Pseudonymes par défaut (`PX-48213`) ; révélation d'identité réservée à certains rôles, journalisée ; **jamais** de PII dans URL, `localStorage`, logs, analytics, messages d'erreur
Cache	`Cache-Control: no-store` sur les pages de données de santé
Dépendances	`npm audit` + Dependabot en CI ; versions figées par lockfile
Transparence IA (AI Act)	Mention « Généré par IA » sur les réponses de l'assistant ; score présenté comme **aide à la décision** ; explication SHAP toujours accessible
Supervision humaine	Aucune action automatique pénalisante : l'alerte est une recommandation, la décision est humaine, motivée et tracée
Contestation	Le professionnel voit le motif d'un rejet ou d'une demande de justificatif et peut répondre ou contester
Équité	Jamais de variable sensible dans les explications affichées ; page admin « Audit d'équité » (métriques par groupe)

12. Supervisabilité et exploitation

- `GET /api/health` (statut de l'application + joignabilité de l'API) pour les sondes Docker ; `HEALTHCHECK` dans le Dockerfile.

- Journaux serveur **structurés JSON** (niveau, route, durée, identifiant de corrélation `x-request-id`), sans PII.
- Écran admin « Supervision » : statut des services, latence p50/p95, volumes traités, taux d'erreur, dérive du modèle ; rafraîchissement automatique avec horodatage de la dernière mise à jour.
- Remontée d'erreurs frontend (Sentry auto-hébergé ou équivalent) avec scrubbing de données.
- Écran de **bascule de modèle** : liste versionnée, comparaison de métriques, retour arrière sans interruption, confirmation + audit.
- Une seule image Docker, configurée par `.env` (documenté dans `.env.example`) ; build reproductible.

13. Performance

- Budgets : JS initial ⩽ 180 Ko gzip par route ; LCP < 2,5 s ; INP < 200 ms ; CLS < 0,1 (Lighthouse CI bloquant).
- Server Components par défaut ; `next/dynamic` pour graphiques et éditeurs lourds ; `next/image` ; polices via `next/font`.
- Listes de plus de 100 lignes : pagination serveur ou virtualisation.
- Pas de bibliothèque ajoutée sans justification de poids (ADR si > 30 Ko).

14. Tests et CI

Niveau	Outil	Seuil
Unitaires (formatage, schémas zod, RBAC)	Vitest	⩾ 80 % sur `lib/` et `features/*/schemas`
Composants	Testing Library + Storybook	Chaque composant `domain` : 5 états testés
E2E par rôle	Playwright	Parcours critiques : dépôt facture, traitement d'alerte, question à l'assistant, changement de seuil, bascule de modèle
Accessibilité	axe-core	0 violation critique/sérieuse

Niveau	Outil	Seuil
Robustesse	Playwright	Lot de 20 factures dont invalides : rejet propre, rapport par facture
Contraste des tokens	script CI	100 % des paires conformes
Régression visuelle	Storybook + Chromatic/Playwright snapshots	PEUT

Pipeline CI (bloquant) : `lint` → `typecheck` → `test` → `build` → `e2e + a11y` → `lighthouse` → build image Docker → scan de vulnérabilités. Merge interdit si l'un échoue.

15. Conventions de code et de Git

- ESLint (next/core-web-vitals, jsx-a11y, import/order) + Prettier ; Husky + lint-staged avant commit.
- Branches `feat/…`, `fix/…`, `docs/…` ; **Conventional Commits** (`feat(alerts): add SHAP waterfall`) ; PR ⩽ 400 lignes, une revue minimale.
- Pas de `console.log` ; utiliser `lib/logger.ts`. Pas de `TODO` sans ticket.
- Composants : props nommées explicitement, pas de prop `any`, pas de logique dans le JSX ; effets (`useEffect`) en dernier recours.
- Imports absolus `@/`. Pas de dépendance circulaire.

16. Documentation

Document	Contenu	Emplacement
`README.md`	Présentation, prérequis, `docker compose up`, variables, architecture, scripts	racine
`docs/adr/NNNN-titre.md`	Décisions d'architecture (contexte, décision, conséquences)	`docs/adr/`
Storybook	Catalogue vivant des composants, usages, do/don't	publié en CI

Document	Contenu	Emplacement
Guide utilisateur par rôle	Captures annotées, tâches pas à pas, FAQ	`docs/user-guide/`
Guide d'exploitation (partie front)	Configuration `.env`, health, logs, mise à jour, incidents courants	`docs/ops/`
`CHANGELOG.md`	Keep a Changelog + SemVer	racine
`CONTRIBUTING.md`	Ce référentiel, workflow Git, Definition of Done	racine
Déclaration d'accessibilité	Niveau de conformité et limites connues	dans l'application

Tout composant partagé : commentaire TSDoc (rôle, props, exemple) + story. Toute décision structurante : un ADR court.

17. Definition of Done et checklist de PR

Une fonctionnalité est **terminée** si :

- [ ] Elle respecte tokens, composants et ton rédactionnel de ce document
- [ ] Les 5 états (chargement, vide, erreur, partiel, nominal) sont gérés
- [ ] Utilisable au clavier, lecteur d'écran vérifié, 0 violation axe
- [ ] Responsive vérifié (320 px, 768 px, 1280 px)
- [ ] Aucun texte en dur, aucune couleur en dur, aucun secret, aucune PII exposée
- [ ] Permissions vérifiées pour chacun des 4 rôles (test e2e)
- [ ] Tests ajoutés et CI verte ; budgets de performance respectés
- [ ] Story Storybook et documentation mises à jour ; ADR si décision structurante
- [ ] Actions sensibles auditées (décision, seuil, modèle, révélation de PII)

18. Alignement avec les 8 conditions « plateforme exploitable » du cahier des charges

Condition	Contribution du frontend
Déployable	Image Docker standalone, `docker compose up`, health check
Utilisable	Interface unique, connexion, 4 rôles, parcours sans ligne de commande
Persistante	Aucun état critique côté navigateur ; tout vient de l'API/PostgreSQL
Configurable	Écran admin du seuil et des canaux, `.env` documenté, changement visible en direct
Supervisable	Écran de supervision (latence, volume, erreurs, dérive), `/api/health`, logs JSON
Maintenable	Sélecteur de version de modèle avec retour arrière, tests et CI
Robuste	Rapport de rejet par facture, états d'erreur explicites, validation zod
Documentée	Guide utilisateur par rôle, guide d'exploitation, Storybook

19. Ordre de mise en œuvre conseillé

1. **Fondations** : projet Next.js, tokens, polices, primitives, Storybook, CI, Dockerfile.
2. **Coque et sécurité** : authentification, RBAC, layouts par rôle, `/api/health`.
3. **Cœur métier** : dépôt de facture, file d'alertes, détail avec SHAP, décision.
4. **Portails** : suivi assuré, suivi professionnel, assistant RAG.
5. **Pilotage** : dashboard BI, écran d'administration (seuil, modèles, supervision, audit).
6. **Durcissement** : accessibilité, performance, tests e2e par rôle, lot de factures invalides.
7. **Documentation et soutenance** : guides, captures, démo scriptée par rôle.

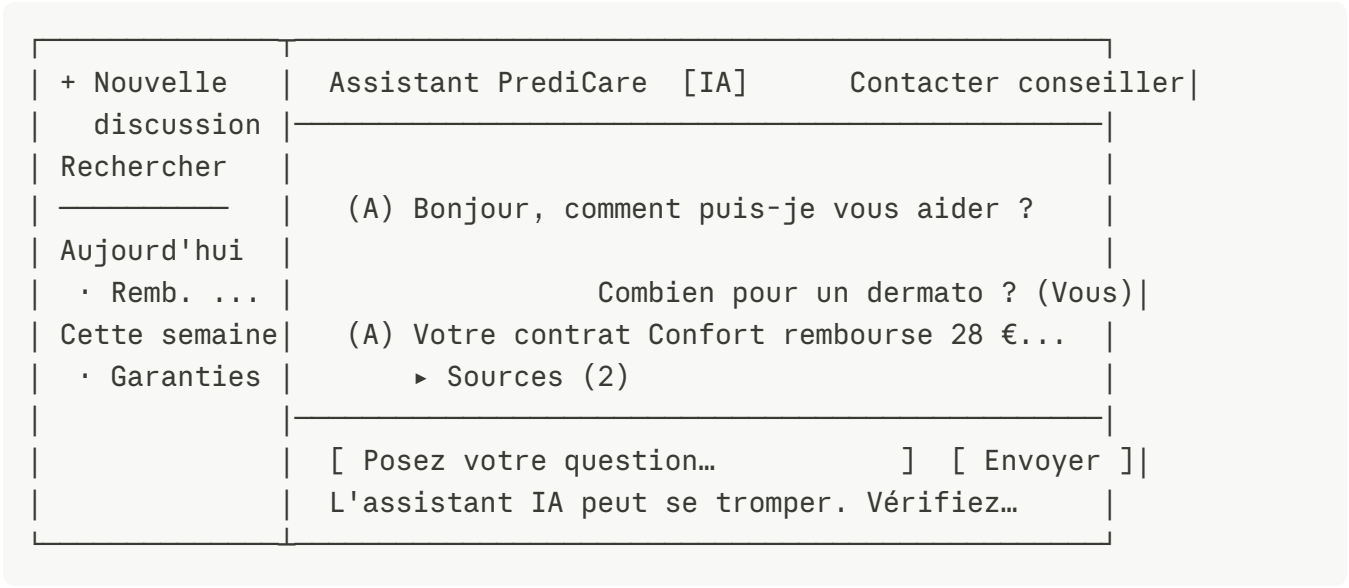
Annexe A — Page Assistant (chatbot) : règles de conception et d'ingénierie

Portée : route `/assistant`. Utilisateur principal : l'assuré (Flux 2 du cahier des charges : Agent ReAct → base vectorielle des contrats + historique SQL → LLM sécurisé → réponse sourcée). Les autres rôles peuvent y accéder avec un périmètre réduit (barèmes et contrats génériques, jamais les données d'un assuré). Cette annexe complète les sections 5 à 11 ; en cas de conflit, c'est elle qui prévaut pour cette page.

A.1 Principes

Principe	Traduction concrète
Une vraie conversation	Fil de messages, bulles, saisie en bas, réponses qui s'écrivent en direct : les codes d'une application de messagerie moderne.
La réponse d'abord	La première phrase donne le montant, le statut ou le oui/non. Les détails suivent.
Du texte propre, jamais de symboles	L'utilisateur ne voit jamais `**`, `#`, `##`, `*`, `---`, des barres de tableau ni des crochets bruts.
Des sources discrètes mais vérifiables	Masquées par défaut, dépliables en un clic, ouvrables à la bonne page du document.
Honnêteté	L'assistant dit qu'il est une IA, dit quand il ne sait pas, et refuse proprement hors périmètre.
Une sortie vers l'humain	Le bouton « Contacter un conseiller » est accessible à tout moment.

A.2 Structure de l'écran



- **DOIT** : la page occupe la hauteur de l'écran (`h-dvh`) ; **seul le fil de messages défile**. En-tête et zone de saisie restent fixes.
- **DOIT** : colonne de lecture centrée, largeur maximale **720 px** ; panneau d'historique de **280 px**, repliable.
- **DOIT** : en-tête avec nom de l'assistant, badge « IA » permanent, menu (renommer, effacer, paramètres) et bouton « Contacter un conseiller ».
- **DEVRAIT** : sous 1024 px, l'historique passe dans un panneau latéral (Sheet) ouvert par un bouton.

A.3 Messages

Élément	Règle
Message utilisateur	Aligné à droite, fond `primary-soft` , texte `foreground` , rayon 12 px (coin bas droit 4 px), largeur max 80 %
Message assistant	Aligné à gauche, avatar 28 px (pictogramme pouls sur fond `navy`), fond `surface` avec bordure `border` , largeur max 100 % de la colonne
Groupement	Messages consécutifs d'un même auteur resserrés (4 px), avatar affiché une seule fois par groupe

Élément	Règle
Horodatage	Au survol et au focus clavier, plus des séparateurs de jour (« Aujourd'hui », « Hier », « 12 mars »)
Statut d'envoi	« Envoi… » puis discret « Envoyé » ; en cas d'échec : message marqué en erreur avec « Réessayer » (le texte n'est jamais perdu)
Actions du message assistant	Copier · Utile · Pas utile · Régénérer (dernier message seulement). Visibles au survol sur desktop, **toujours visibles** sur mobile
Copier	Copie du **texte brut propre**, sans symboles ni marqueurs de sources
Message d'accueil	Présente le périmètre, rappelle qu'il s'agit d'une IA, propose 4 suggestions cliquables adaptées au rôle

A.4 Ressenti en direct (streaming)

- Le message de l'utilisateur apparaît **immédiatement** (< 100 ms), suivi d'un indicateur de frappe (trois points animés, désactivés si `prefers-reduced-motion`).
- Pendant que l'agent travaille, afficher son état **en langage humain**, jamais en termes techniques :

Étape de l'agent (événement `status`)	Libellé affiché
`contract_search`	Je consulte votre contrat…
`account_lookup`	Je vérifie votre dossier…
`composing`	Je rédige la réponse…

- La réponse arrive en flux (SSE), mot après mot, avec un rendu limité à ~30 mises à jour par seconde. Le libellé d'état disparaît au premier mot.
- Pendant la génération, le bouton « Envoyer » devient **« Arrêter »** ; la touche Échap fait de même. Le texte déjà reçu est conservé et marqué « Réponse interrompue ».
- **Défilement automatique** uniquement si le lecteur est déjà en bas (tolérance 80 px). Sinon, une pastille « Nouveau message ↓ » apparaît ; elle ne déplace jamais la lecture en cours.

- Pendant le flux, aucun symbole en attente n'est affiché : un gras ou une liste non terminés sont rendus proprement, jamais avec leurs astérisques.

A.5 Mise en forme du texte (règle centrale)

Règle d'or : ce que l'utilisateur lit est du texte naturel, comme dans un message écrit par une personne. Deux lignes de défense, toutes deux obligatoires.

1. Consigne de style côté backend (à intégrer au prompt système) :

```
Tu es l'assistant de PrediCare. Réponds en français, en vouvoyant, sur un ton chaleureux et sobre.
Commence par la réponse directe. Écris 2 à 4 paragraphes courts, sans titres, sans emojis.
N'utilise aucun symbole de mise en forme, sauf des tirets pour une liste de 3 éléments ou plus.
Cite tes sources uniquement avec les identifiants fournis, par exemple [S1].
Si l'information est absente des documents ou du dossier, dis-le et propose un conseiller.
Ne donne jamais d'avis médical. Présente les montants comme des estimations.
```

2. Rendu côté frontend (composant `FormattedText`) :

- Le texte passe par un parseur Markdown **à liste blanche** qui produit de vrais éléments, jamais du texte avec symboles.
- Éléments autorisés : paragraphes, gras discret (graisse 600, même couleur), italique, listes à puces et numérotées, liens internes ou sur liste d'autorisation, tableau simple (comparaison à 3 lignes ou plus uniquement).
- Éléments interdits : titres `h1` à `h6` (convertis en paragraphe), citations, blocs de code, images, HTML brut, lignes de séparation.
- **Assainisseur de secours** (`strip-markdown-fallback.ts`) : si un symbole subsiste après le rendu (`**`, `#` en début de ligne, `` ` ``, `---`, `|---|`), il est supprimé ou converti. Un test automatisé vérifie qu'aucun de ces motifs n'apparaît dans le DOM final.
- `dangerouslySetInnerHTML` reste interdit.

Typographie du fil : corps 15 px (16 px sur mobile), interligne 1,6, 12 px entre paragraphes, listes avec retrait de 20 px. Formats français : `42,50 €`, `12 mars 2026`, `70 %` (espace insécable), via `Intl`.

Style éditorial

Règle	Détail
Réponse d'abord	Première phrase = montant, statut ou oui/non
Court par défaut	120 mots maximum ; plus long seulement sur demande
Listes avec parcimonie	Seulement pour 3 éléments parallèles ou plus
Ton humain	Vouvoiement, chaleureux et sobre ; pas de « En tant qu'IA », pas de reformulation de la question, pas d'excuses répétées
Précision et conditions	Montants et dates exacts, avec la condition utile (« selon votre contrat Confort »)
Estimation assumée	Un remboursement est annoncé comme une estimation, jamais comme une promesse
Une seule suite	Terminer par une action utile ou une seule question de clarification
Incertitude	Dire ce que l'on sait, ce qui manque, puis proposer un conseiller
Pas d'avis médical	Aucune recommandation de santé

Exemple fictif

À proscrire (ce que l'utilisateur ne doit jamais voir) :

```
## Remboursement dermatologue
**Taux :** 70% de la BR
- Plafond : 50€
Source : [1][2]
```

Attendu (texte affiché) :

```
Pour une consultation à 40 € chez un dermatologue, votre contrat Confort vous
rembourse environ 28 €, soit 70 % de la base de remboursement.

Les dépassements d'honoraires éventuels ne sont pas couverts par ce niveau de
garantie. Il s'agit d'une estimation : le montant exact apparaît dans votre
décompte une fois le dossier traité.

Souhaitez-vous que je vérifie le remboursement de votre dernière consultation
?
```

A.6 Références (sources)

Principe : masquées par défaut, dépliables à la demande, toujours vérifiables.

- **Marqueurs dans le texte** : discrets, en exposant, sous forme de pastilles numérotées de 16 px (1, 2…), à la place de `[S1]`. Le survol ou le focus affiche une bulle avec le titre du document et la page.
- **Ligne repliée sous chaque réponse** : « Sources · 2 documents » avec icône et chevron. Un seul bouton, `aria-expanded` et `aria-controls`.
- **Dépliée**, une carte par source :

Champ	Contenu
Type	Icône « Contrat » ou « Votre dossier »
Titre	`Contrat Confort — Conditions générales`
Localisation	`Article 4.2 · page 12`
Extrait	200 caractères maximum, passage pertinent surligné
Actions	« Ouvrir à la page 12 » (panneau latéral avec visionneuse PDF chargée à la demande) · « Copier la référence »

- Pour les données du compte, la source indique la nature et la date, sans donnée sensible superflue : `Votre dossier · Remboursement du 12/03/2026 · Facture FAC-2026-00481`.
- Un clic sur un marqueur déplie la ligne, fait défiler jusqu'à la carte correspondante et la met en évidence 1,5 s.

- **Préférence utilisateur** « Toujours afficher les sources » (désactivée par défaut), enregistrée côté serveur.
- **Intégrité** : les sources viennent **uniquement** de l'événement `sources` de l'API (identifiants résolus côté serveur). L'interface n'extrait jamais une référence ou un lien du texte généré. Un marqueur sans source correspondante est supprimé.
- **Réponse sans source** : si une réponse factuelle arrive sans aucune source, un badge ambre « Réponse non sourcée » s'affiche et l'action « Contacter un conseiller » est mise en avant.
- Accès aux documents : contrats de l'assuré connecté et barèmes génériques uniquement, contrôlé par l'API.

A.7 Zone de saisie

- Champ qui s'agrandit de 1 à 6 lignes, puis défile. Placeholder : « Posez votre question sur vos remboursements ou vos garanties ».
- **Entrée** envoie, **Maj + Entrée** crée une ligne ; la composition IME (clavier asiatique, accents) n'envoie jamais par erreur.
- Bouton « Envoyer » désactivé si le champ est vide ; **2 000 caractères maximum**, compteur visible à partir de 80 %.
- Collage nettoyé (texte brut). Après l'envoi, le focus reste dans le champ.
- Pendant la génération : bouton « Arrêter ». L'envoi d'un nouveau message est possible après l'arrêt ou la fin de la réponse.
- Mention permanente sous le champ, en `muted-foreground` 12 px : « L'assistant IA peut se tromper. Vérifiez les informations importantes dans votre contrat. »
- Suggestions cliquables (4 maximum) sous l'accueil, adaptées au rôle : « Où en est mon dernier remboursement ? », « Que couvre mon contrat pour l'optique ? »…
- Brouillon conservé **en mémoire pour la session uniquement**, jamais dans `localStorage`.

A.8 États et microcopy

État	Message affiché	Action
Premier usage	Bonjour, je suis l'assistant PrediCare. Je peux vous aider à suivre vos remboursements et à comprendre vos garanties.	4 suggestions

État	Message affiché	Action
Hors périmètre	Je ne peux pas répondre à cette question, mais je peux vous aider pour vos remboursements et vos garanties.	Suggestions
Information introuvable	Je n'ai pas trouvé cette information dans votre contrat. Un conseiller pourra vous répondre.	Contacter un conseiller
Demande bloquée (sécurité)	Je ne peux pas traiter cette demande. Vous pouvez reformuler votre question sur vos remboursements ou vos garanties.	Reformuler
Réseau coupé	Connexion interrompue. Votre message n'est pas perdu.	Réessayer
Réponse lente (> 30 s)	La réponse prend plus de temps que prévu.	Attendre · Annuler
Trop de messages	Vous envoyez beaucoup de messages. Réessayez dans quelques secondes.	Compte à rebours
Hors ligne	Bandeau : Vous êtes hors ligne.	Saisie désactivée
Session expirée	Votre session a expiré. Reconnectez-vous pour continuer.	Retour connexion, brouillon conservé

Les messages de blocage de sécurité ne détaillent jamais la raison technique (pas d'indice pour un attaquant).

A.9 Historique des conversations

- Stockées **côté serveur** (PostgreSQL), isolées par utilisateur ; jamais dans le navigateur.
- Liste groupée par période (Aujourd'hui, 7 jours, plus ancien), titre généré automatiquement, renommable.
- Actions : nouvelle discussion, recherche, renommer, supprimer (confirmation obligatoire), « Effacer tout l'historique » dans les paramètres (droit à l'effacement).
- Durée de conservation affichée dans les paramètres. Toute suppression est journalisée sans contenu.

- Reprise d'une conversation : les sources restent consultables telles qu'elles étaient à la date de la réponse.

A.10 Retour utilisateur et escalade

- « Utile » : enregistrement immédiat. « Pas utile » : puces de motif facultatives (Incorrect · Incomplet · Pas clair · Autre) et zone libre.
- « Contacter un conseiller » : dialogue qui résume la conversation, demande confirmation avant transmission et indique le délai de réponse attendu.
- Le retour utilisateur alimente le suivi qualité de l'assistant (écran admin) sans le texte des conversations par défaut.

A.11 Sécurité, RGPD et AI Act spécifiques

- **Aucun HTML** issu du modèle n'est exécuté ; images distantes, scripts et iframes interdits ; liens sortants limités à une liste d'autorisation.
- L'interface **n'envoie jamais d'identifiant d'assuré** : l'API le déduit de la session. Aucun paramètre de requête ne peut changer le périmètre de données.
- Les textes saisis et reçus ne sont **pas** envoyés aux outils d'analytique et n'apparaissent pas dans les journaux frontend, les URL ou les rapports d'erreur.
- Défense en profondeur contre l'injection de prompt : limites de longueur et nettoyage à la saisie ; le blocage réel est côté serveur ; message neutre en cas de refus.
- Transparence (AI Act) : badge « IA » permanent, mention sous le champ de saisie, accès humain constant.
- Aucune décision de remboursement n'est prise par l'assistant : il informe et estime.
- Cache désactivé (`no-store`) pour cette page et ses appels.

A.12 Accessibilité

- Le fil de messages est un `role="log"` avec `aria-live="polite"` : la réponse est annoncée **une seule fois, à la fin**, pas mot par mot ; `aria-busy` pendant la génération.
- Raccourcis : `/` place le focus dans la saisie, `Échap` arrête la génération, `Alt + N` ouvre une nouvelle discussion ; ils sont listés dans une aide (`?`).
- Ordre de tabulation : en-tête → historique → messages (actions) → saisie. Focus visible partout, aucun piège clavier.
- Sources : motif « disclosure » WAI-ARIA, marqueurs atteignables au clavier, alternative textuelle à toute icône.

- Contrastes AA ; animations (points, apparition) désactivées si `prefers-reduced-motion`.
- Zoom 200 % et reflow 320 px sans perte.

A.13 Responsive mobile

- Plein écran, historique dans un Sheet, zone de saisie collée au clavier virtuel (`dvh`, `visualViewport`, `safe-area-inset-bottom`).
- Actions de message toujours visibles ; cibles tactiles ⩾ 44 px ; police 16 px dans le champ pour éviter le zoom automatique.
- Les sources s'ouvrent dans un panneau plein écran.

A.14 Architecture et contrat d'API

```
features/assistant/
├─ components/  ChatLayout, ConversationList, MessageList, UserMessage,
│                AssistantMessage, FormattedText, TypingIndicator,
AgentStatus,
│                CitationMarker, SourcesDisclosure, SourceCard,
PdfSourceSheet,
│                Composer, SuggestionChips, MessageActions, FeedbackPopover,
│                HandoffDialog, EmptyWelcome, ErrorBanner
├─ hooks/       useChatStream, useAutoScroll, useConversations,
useComposerState
├─ lib/         markdown-whitelist.ts, strip-markdown-fallback.ts, format-
fr.ts
├─ schemas/     message.ts, stream-event.ts   (zod)
├─ api.ts
└─ index.ts
```

```
interface Source {
  id: string;
  kind: 'contrat' | 'dossier';
  title: string;
  locator: string;      // ex. « Article 4.2 · page 12 »
  page?: number;
  excerpt: string;      // 200 caractères max
  documentId?: string;
}

interface ChatMessage {
  id: string;
  role: 'user' | 'assistant';
  text: string;
  sources: Source[];
  status: 'sending' | 'streaming' | 'done' | 'interrupted' | 'error';
  createdAt: string;
  feedback?: 'up' | 'down';
}
```

Flux SSE (`POST /assistant/conversations/{id}/messages`), chaque événement validé par `zod` :

```
event: status   data: {"step":"contract_search"}
event: token    data: {"text":"Pour une consultation "}
event: sources  data: {"items":
[{"id":"S1","kind":"contrat","title":"...","page":12}]}
event: done     data: {"messageId":"m_123"}
event: error    data: {"code":"RATE_LIMIT","retryAfter":20}
```

- Un événement invalide est ignoré et journalisé ; une coupure en cours de flux produit l'état `interrupted` avec « Réessayer ».
- Le client ne reconstruit jamais l'état à partir du texte : il consomme les événements typés.

A.15 Performance

- Composants de message mémoïsés ; seul le dernier message se met à jour pendant le flux.
- Virtualisation au-delà de 200 messages ; chargement à la demande de l'historique ancien.

- Visionneuse PDF et parseur Markdown chargés dynamiquement ; premier message affiché en moins de 200 ms après l'envoi.
- Aucun re-rendu de la liste d'historique pendant le flux.

A.16 Tests et Definition of Done spécifiques

Test	Critère
Rendu propre	Corpus de 30 réponses types (gras, titres, listes, tableaux, flux partiels) : aucun `**`, `#`, `` ` ``, `---` visible dans le DOM
Sources	Repliées par défaut ; dépliables au clic et au clavier ; marqueur orphelin supprimé ; réponse sans source = badge
Flux	Arrêt manuel, coupure réseau en cours de flux, erreur SSE, réponse lente, limite de débit
Défilement	Pas de saut quand l'utilisateur a remonté ; pastille « Nouveau message »
Saisie	Entrée / Maj+Entrée, IME, limite de 2 000 caractères, collage nettoyé
Sécurité	Tentative d'injection : message neutre, aucun HTML exécuté, aucun lien hors liste
Isolation	Un assuré ne voit jamais l'historique ni les sources d'un autre (test e2e à deux comptes)
Accessibilité	axe : 0 violation ; lecteur d'écran : annonce unique à la fin de la réponse
Mobile	Saisie visible avec clavier ouvert à 320 px

Une évolution de l'assistant est **terminée** si :

- [ ] Le texte affiché ne contient aucun symbole de mise en forme brut
- [ ] Les sources sont masquées par défaut, dépliables et ouvrent la bonne page
- [ ] Les états vide, chargement, flux, interrompu, erreur, hors périmètre et sans source sont traités
- [ ] « Contacter un conseiller » et la mention IA sont présents

- [ ] Aucune donnée de conversation dans les journaux, URL, analytique ou `localStorage`
- [ ] Tests, accessibilité et responsive validés ; documentation et Storybook mis à jour